- To check current working directory
    - getwd()
- To change current working directory
    - setwd()
- Assigning new objects to the environment
    - Object_name <-
    - After you assign, state the object name again underneath for it to show up
- round(variable, number of digits to be round to)
- c() is combine and create a vector
    - mean() for avg
- class()
    - To figure out what type of data
- is()
    - is.integer(my_numeric)
    - Asking if the “my_numeric is an integer
    - my_interger <- as.integer(my_numeric)
        - Converting numeric to integer
    - is.na()
        - Is this value missing
    - any.na()
        - Any value missing
- na.rm=TRUE
    - Removes NA values
- Create vector
    - My_vector <- c(chr or numeric)
    - [] pulls out an element from a vector
- Data frames
    - tibble() creates new data frame
    - names() gives variable names in column
    - For dimensions of data frame
        - dim()
        - str()
        - [#,#] -> row, column
    - $
        - Access a column (variable) in a data frame
        - mean(data$column) calculates mean
    - Writing data to file
        - write.csv(data, “data.csv”, row.names = FALSE)
- Read.csv reads data into new name/variable
- as.factor(data$columns)

    - Changes type for the column in the data
    - levels() -> levels for column
- Summary
    - glimpse()
    - summary()
    - Nrow, ncol, dim, names
    - head() gives first n rows of data frame
    - tail() gives last n rows
- select()
    - Used to extract columns
    - Range of columns -> start_col:end_col
    - !
        - Select everything except specified variable
        - !c(), choose multiple columns to exclude
- rename(data, new_name = old_name)
- filter()
    - Extract rows
    - Can use conditions
    - filter()` allows all of the expected operators; i.e. >, >=, <, <=, != (not equal), and == (equal).
        - filter(penguins, **species!=**"Chinstrap")
    - %in%
        - Determines whether or not a value is part of a vector
    - between()
        - Range of specific values
    - How would you use `%in%` to get the same result?
        - filter(penguins, body_mass_g %in% c(5250, 5300, 5350, 5400, 5450, 5500))
    - Filter the fish data to include all fish with a scalelength within 0.25 of 8.
        - filter(fishes, near(scalelength, 8, tol=0.25))
    - | is or either condition occurs.
    - + `filter(condition1, condition2)` will return rows where both conditions are met. By default the , means &.
    - + `filter(condition1, !condition2)` will return all rows where condition one is true but condition 2 is not.
    - + `filter(condition1 | condition2)` will return rows where condition 1 or condition 2 is met.
    - . We are interested in the fish from the lakes "AL" and "AR" with a radii length between 2 and 4. Extract this information from the `fish` data. Please sort the data in descending order by radii length.
        - fishes %>%

        - select(lakeid, radii_length_mm) %>%
        - filter(lakeid=="AL"| lakeid=="AR") %>%
        - filter(between(radii_length_mm, 2, 4)) %>%
        - arrange(desc(radii_length_mm))
- homerange$mean.hra.m2 <- as.numeric(homerange$mean.hra.m2)
    - filter(homerange, class == "mammalia", trophic.guild == "herbivore", mean.hra.m2>1000000) %>%
    - select(common.name, order, mean.hra.m2, mean.mass.g) %>%
    - arrange(desc(mean.hra.m2)) %>%
    - slice_max(mean.hra.m2, n=10)
- distinct()
    - Acts on rows
- mutate()
    - Acts on columns
        - Create new columns from existing ones
    - penguins %>%
        - mutate(body_mass_kg=body_mass_g/1000) %>%
        - select(species, body_mass_g, body_mass_kg) %>%
        - arrange(body_mass_kg)
    - Use across() to apply to multiple columns
        - So, `./10` means "take the current column and divide it by 10". This operation is applied to all columns that end with `mm`.
    - mammals %>%
        - mutate(across(where(is.character), tolower)) #only variables that are class character to lower case

      ~ifelse(.==-999, NA,.)

        Replaces -999 with NA
- !is.na gives no NA's
    - This goes in filter
- na.rm = T
    - Goes in summarize
- summarize(across(where(is.numeric),
- ~mean(.x, na.rm = T))) #.x = every cell, ~ = function
    - ^calculates mean of every numeric column in a data frame, ignoring missing values
- Summarize best used with group_by()
    - Allows to group data by a categorical variables of interest
- We can group by more than one variable.
    - group_by(sex, years_of_sample_collection) %>%

    - summarize(mean_estimated_age_years =
      mean(estimated_age_years, na.rm = T),
    - n=n(), #n is number of samples
    - .groups='keep') %>% #removes the warning message
    - arrange(sex)
- any(is.na(fisheries_clean))
    - Looks for na values
- How many countries are represented in the data?
    - summarize(num_country = n_distinct(country))
- How has fishing of this species changed over the last decade (2013-2023)? Create a plot showing total catch by year for this species.
    - fisheries_clean %>%
    - filter(common_name == "Alaska pollock(=Walleye poll.)",
    - period >= 2013,
    - period <= 2023) %>%
    - group_by(period) %>%
    - summarize(overall_catch = sum(catch, na.rm = T)) %>%
    - ggplot(aes(x = period, y = overall_catch))+
    - geom_point()+
    - geom_smooth(method="lm", se = TRUE)+
    - labs(title = "Total Alaska Pollock Catch (2013-2023)", x = "Year", y = "Total Catch")
- + `pivot_longer`(cols, names_to, values_to)
- + `cols` - Columns to pivot to longer format
- + `names_to` - Name of the new column; it will contain the column names of gathered columns as values
- + `values_to` - Name of the new column; it will contain the data stored in the values of gathered columns
    - pivot_longer(-religion,
    - names_to ="income", #give new column name
    - values_to = "total") #move the count into new column
- billboard %>%
    - pivot_longer(
    - cols=starts_with("wk"),
    - names_to = "week",
    - names_prefix = "wk", #gets rid of the wk
    - values_to = "rank",
    - values_drop_na = T
    -)
- qpcr_tidy %>%

        - separate(experiment, into=c("experiment", "e_number"), sep=3) %>% **this splits exp into 2 columns and sep=3 means split after the 2rd character**
        - separate(replicate, into=c("rep", "r_number"), sep=3) %>%
        - select(gene, experiment=e_number, replicate=r_number, `mRNA expression`) **renaming**
    - 2. Restrict the data to medical and health expenditures only. Sort in ascending order.
        - pivot_longer(-expenditure,
        - names_to = "year",
        - values_to = "bn_dollars") %>%
        - filter(expenditure=="Medical and Health") %>%
        - arrange(bn_dollars)
    - unite()
        - Give column name and list of columns to combine with separation character
    - + `pivot_wider`(names_from, values_from)
    - + `names_from` - Values in the `names_from` column will become new column names **identify variables or new column names and values_from to identify values from those specific columns**
    - + `values_from` - Cell values will be taken from the `values_from` column
    - pivot_longer(-year,m)# this means to exclude the column
        - You want to make a ggplot
          You want to facet by a variable
          You want categories stacked in one column
          Your column names are actually data
- Which month usually has the highest average enterococci levels across all beaches?
        - group_by(month) %>%
        - summarize(avg_enterococci = mean(enterococci_cfu_100ml, na.rm = T)) %>%
        - arrange(desc(avg_enterococci))
    - Geom_bar
        - Count the number of observations in a categorical variable
    - Geom_col
        - Used when you already count the number of observations
        - Have a defined x and y and you want to plot these counts
    - group_by(order) %>%
        - summarize(n=n())
        - A shorter form is

        - count(order, sort = t)
        - Counts how many times each unique value appears in the order column, then sorts from highest to lowest frequency
- In ggplot(aes(x=reorder(order, mean_mass), y=mean_mass))
    - Adjust x-axis labels to make them more readable
- We can adjust the plot further by specifying the size and face of the text. We do this using `theme()`. The `rel()` option changes the relative size of the title to keep things consistent. Adding `hjust` controls the title position.
    - theme(plot.title=element_text(size=rel(1.5), hjust=.5))
- Play with point size by adjusting the `size` argument.
    - p+geom_point(size=1.5) #geom_jitter helps with overplotting
- We can color the points by a categorical variable.
    - p+geom_point(aes(color=thermoregulation), size=1.5)
- We can also map shapes to another categorical variable.
    - p+geom_point(aes(shape=thermoregulation, color=thermoregulation), size=1.5)
- What about just filling in with a color preference or size?
    - p+geom_point(size=3, color="black", alpha=0.8, shape=24, fill="blue")# color is the outline and fill is the color on the inside
- group() groups data for plotting
    - Doesnt add color
- scale_y_continuous(labels=scales::percent)
    - Formats y-axis labels as percentages
- position=”dodge”
    - Places bar or column plots next to each other
- “Are my categories stored as values in one column?”
- Plot categories
  → That’s LONG.
- “Are my categories stored as separate columns?”
  → That’s WIDE, do math bw categories
- injury!= “fatal”
    - Keeps row where injury is not exactly the text fatal
- ifelse(is.na(minor), 0, minor)
    - If minor is NA → use 0
    - Otherwise → use the actual value in minor
    - Treats missing values as 0
- p+theme_linedraw()+
- theme(legend.position = "bottom",
- axis.text.x = element_text(angle = 60, hjust=1))
        - Makes the plot clean and black-and-white

        - Moves the legend to the bottom
        - Rotates x-axis labels 60° so they don't overlap
- +`scale_colour_brewer()` is for points
- +`scale_fill_brewer()` is for fills
- barplot(rep(1,7), axes=FALSE, col=my_palette)
    - 7 equal-height bars
    - With no axes
    - Each bar colored using `my_palette`
- `facet_wrap()` makes a ribbon of panels by a specified categorical variable and allows you to control how you want them arranged.
    - **Think of `facet_wrap()` as:**
    - "Split by ONE variable and wrap panels like tiles."
    - If `thermoregulation` has 4 categories, you get 4 small plots arranged in 2 columns.
- `facet_grid()` allows control over the faceted variable; it can be arranged in rows or columns. Rows~columns.
    - Comparison of two categorical variables
    - **Think of `facet_grid()` as:**
    - "Split by TWO variables and organize in a matrix."
- summarize(across(everything(), ~sum(is.na(.))))
    - Summary of the number of na's
- Let's use `mutate()` and use `na_if()` to replace 0's with NA's
- miss_var_summary()
    - How many missing values per variable
    - What percentage of column is missing
- Full_join
    - keeps all the rows in both tables and fills in with NA where there is no match
- anti_join()
    - Provides the rows in the first table for which there are not matching values in the second table
- So, `inner_join` only keeps the rows that have matching values in both tables
- So, `left_join` keeps all the rows in the first table and only the matching rows in the second table
- `right_join` does the opposite of `left_join`. It keeps all the rows in the second table and only the matching rows in the first table. If there is no match, it fills in with NA
- mutate(across(where(is.numeric), ~ifelse(. < 0, NA, .)))
    - Any negative number becomes NA

    - Positive numbers and zero stay the same
- gg_miss_var() +
- theme(axis.text.y = element_text(size=7))
    - Creates a bar plot showing missing values per variable
    - Makes the y-axis variable names smaller so they fit better
- Default openstreet maps
    - addTiles()%>%
    - addcirclemarkers))
- 1. The `ui` function controls the user inputs and the way the app will be displayed; i.e. this controls how your app looks.
- 2. The `server` function is the part of the app which takes the values of the user inputs, performs calculations and/or makes plots, and prepares the outputs for display; i.e. this controls how your app works.
- 3. The `run` function combines the ui and server function to run the app.
- `reactive()` creates a **reactive expression** in Shiny.
- A reactive expression **automatically re-runs whenever its inputs change**.
- You can think of it like a **dynamic variable** that updates whenever the user changes something in the UI.
- `input$x` = value of the X-axis selectInput
- `input$y` = value of the Y-axis selectInput
- So we have our inputs in a reactive environment, but we want to actually use those inputs to make a plot and display it on the ui. To make and display the plot, we need to save it to a named output object that the ui can use. To do this we use the reactive expression `renderPlot()` and access the plot on the ui side with `plotOutput()`. The inputs from `selectInput()` are character strings, so we need to use `aes_string()` in `ggplot`.
- Vore != = “NA” -> removes rows where vore is the text “NA”
- !is.na(vore) -> remvoes rows where vore is missing/na
- Dynamics in shiny app
    - X axis: when you want users to group by different categorical variables, eg: vore, order, status
    - Y axis: when you want users to measure/compare
- `.data` → tells dplyr/ggplot: “look inside the data frame for a column”

- `[[input$y]]` → programmatically selects the column named by the string in `input$y`
    - So if `input$y = "sleep_total"` → `.data[[input$y]]` becomes `msleep[["sleep_total"]]`

    - Now ggplot knows which numeric column to plot.
- What if you want to turn on the sidebar and put the inputs there? We can do that by moving the `selectInput()` functions to the `dashboardSidebar()`.